Tariff rules

Item	Rule
First 100 units	Free energy charge
Units 101–200	■2.25 per unit in this slab
Above 200 units	■4.50 per unit above 200
Fixed charge	■50
Meter counter	16-bit range: 0–65535
Peak hours	6 PM–10 PM; rate is 1.20 × normal rate

Submission note: The companion ZIP contains Task01–Task12 source folders and sample outputs. Compile and run the programs on your own computer, check results, and replace sample output with genuine run screenshots/evidence if required. Fill in your register number before submitting.

Task 1: Understand the Problem and Identify Data

Inputs include previous/current meter readings and household consumption. Constants include the 100-unit free limit, 200-unit slab boundary, ■2.25 and ■4.50 rates, ■50 fixed charge, and 65535 meter maximum. Use unsigned integer types for non-negative readings, integer paise for money, and strings for labels/messages. Assumptions: readings are valid and the stated tariff rules apply. Basic billing uses O(1) time and O(1) extra space.

Task 2: Meter Rollover

A 16-bit counter wraps after 65535. Unsigned arithmetic can calculate the difference modulo 65536. The exact-width type uint16_t from is preferable when available.

```
typedef unsigned short meter_t;
unsigned int consumption(meter_t previous, meter_t current) {
    return (unsigned short)(current - previous);
}
```

Tests: 1200 → 1530 gives 330 counts. 65400 → 120 gives 256 counts after rollover: 65535 – 65400 + 1 + 120. Confirm the type width on the target compiler.

Task 3: Slab-Based Billing

Compute charges progressively and add the fixed charge. Integer paise avoids floating-point currency inaccuracies.

```
energy_paise = 0;
if (units > 100)
    energy_paise += (min(units, 200) - 100) * 225;
if (units > 200)
    energy_paise += (units - 200) * 450;
total_paise = 5000 + energy_paise;
```

Units	Energy charge	Fixed charge	Total
0	■0.00	■50.00	■50.00
100	■0.00	■50.00	■50.00
150	■112.50	■50.00	■162.50
200	■225.00	■50.00	■275.00
350	■900.00	■50.00	■950.00

Check the handout's exact slab interpretation and expected outputs before final submission.

Task 4: Macros and Safer Alternatives

Macros perform textual substitution. Parenthesize parameters and the entire expression. Inline functions are usually safer because arguments are evaluated once and types are checked.

```
#define SQUARE(x) ((x) * (x))
static inline int square_int(int x) { return x * x; }
```

These arithmetic operations use O(1) time and O(1) extra space. Avoid macro arguments with side effects.

Task 5: Header Files and Debugging

Put shared declarations in a header with include guards; keep implementation in a source file. Conditional compilation can enable debugging output.

```
#ifndef BILLING_H
#define BILLING_H
long calculate_bill_paise(unsigned int units);
#endif
```

Compile with gcc -std=c11 -Wall -Wextra task05.c -o task05. For debug builds, add -DDEBUG if the program has #ifdef DEBUG blocks.

Task 6: Formatted Input and Output

Match format specifiers to variable types and check scanf's return value. Use %u for unsigned int, %d for int, and %.2f for a floating-point display.

```
unsigned int units;
printf("Enter units: ");
if (scanf("%u", &units) == 1)
    printf("Units consumed: %u\n", units);
else
    printf("Invalid input\n");
```

For robust input, fgets() with strtol() allows more complete validation than unchecked scanf(). Reject malformed or out-of-range readings.

Task 7: Type Conversion

Integer division discards the fractional part. Convert an operand before division when a decimal result is required.

```
int a = 5, b = 2;
printf("%d\n", a / b);          /* 2 */
printf("%.2f\n", (double)a/b); /* 2.50 */
```

Simple arithmetic examples use O(1) time and O(1) extra space. Integer paise is recommended for bill totals.

Task 8: Compilation Stages

The compilation pipeline has preprocessing, compilation to assembly, assembling to object code, and linking.

```
gcc -E task08.c -o task08.i
gcc -S task08.c -o task08.s
gcc -c task08.c -o task08.o
gcc task08.o -o task08
```

Save actual terminal output or screenshots from your own system if compilation evidence is required.

Task 9: Complete Billing Test

Combine input validation, consumption calculation, rollover handling, and slab billing. Test normal readings, rollover, zero consumption, slab boundaries, and invalid input. A simple rollover formula assumes at most one rollover between readings.

Test	Condition	Expected check
1	0 units	■50.00
2	100 units	■50.00
3	150 units	■162.50
4	200 units	■275.00
5	65400 → 120	256 counts
6	Malformed input	Reject and show error

Task 10: Compare Accumulated Totals

Compare integer-paise totals with double-based totals. Floating-point values may have representation errors; integer paise avoids these for amounts expressed in whole paise. Use a sufficiently wide integer type such as long long and check overflow. At ■162.50 per household: 4,000 households total ■650,000; 60,000 total ■9,750,000; 800,000 total ■130,000,000. These are arithmetic examples, not measured billing data.

Task 11: Final Design Recommendation

Keep tariffs in named constants or a validated configuration structure; store money in integer paise; validate readings; and separate consumption, tariff calculation, and output into functions. For runtime tariff changes, load validated configuration rather than recompiling. Billing one household is O(1); processing N households is O(N).

Task 12: Smart-Meter Design Challenge

The challenge uses 35,040 readings per household per year (every 15 minutes), peak hours from 6 PM–10 PM, a 20% tariff increase, and runtime tariff changes. Store timestamped Wh readings, convert Wh to kWh, identify peak periods, look up the tariff valid for the interval, calculate charges, and aggregate the bill.

```
interval_kwh = watt_hours / 1000.0;
effective_rate = normal_rate;
if (is_peak_interval)
    effective_rate = normal_rate * 1.20;
interval_charge = interval_kwh * effective_rate;
```

Example: 1000 Wh = 1 kWh. At a normal rate of ■2/kWh, a peak interval costs ■2.40 before other charges. A real system also needs timestamp/time-zone handling, tariff versioning, missing-reading rules, a rounding policy, secure storage, and aggregation. Keeping every yearly reading uses O(N) storage; streaming aggregation uses O(N) time and O(1) working storage if raw readings are archived separately.

Task : Conclusion

The assignment covers problem analysis, C data types, rollover arithmetic, slab billing, macros, headers, input/output, conversions, compilation stages, testing, numerical accuracy, and smart-meter design. Before submission, compile every program, resolve warnings, verify all outputs, add genuine execution evidence, fill in student details, and follow the instructor's exact repository and report requirements.